COMSATS UNIVERSITY ISLAMABAD Abbottabad Campus DEPARTMENT OF COMPUTER SCIENCE ---

MID-TERM AI PROJECT REPORT

AI-BASED CYBER ATTACK PATH SIMULATION SYSTEM

Submitted By: Muhammad Azan Jehangir

Registration Number: CIIT/SP23-BSE-125/ATD

Submitted By: Nouman Dada

Registration Number: CIIT/FA23-BCS-111/ATD

Submitted To: Ms. Zeenat Zulfiqar

Degree Program: Bachelor of Science in Software Engineering (2023–2027)

TABLE OF CONTENTS & SUMMARY

Table of Contents

1. Executive Summary
2. Problem Formulation
3. System Architecture & Topology Mapping
4. Uninformed Search Strategies (BFS & DFS)
5. Informed Search Strategies (UCS & A*)
6. Local Search & Optimization (Hill Climbing)
7. Adversarial Game Search Modeling (Minimax & Alpha-Beta)
8. Empirical Results Matrix
9. Comparative Discussion & Analysis

10. Project Conclusion

1. Executive Summary

This project presents an artificial intelligence framework designed to model enterprise infrastructure as a complex directed graph to simulate cyber attack paths. By treating assets as state nodes and defensive configurations as step-costs, we analyze how different search spaces behave when exposed to systematic threats. This report outlines the traversal properties, optimality limitations, and state-expansion efficiencies of seven core AI pathfinding algorithms running on a customized network topology.

: PROBLEM FORMULATION & SYSTEM TOPOLOGY

2. Problem Formulation

In graph-based threat modeling, enterprise architecture is mathematically defined as a directed, weighted graph $G = (V, E)$, where:

- **Vertices (V):** Represent hosts, gateways, subnets, or distinct targets.
- **Directed Edges (E):** Represent communication paths or accessibility routes between assets.
- **Weights (W):** Represent resistance barriers, such as network difficulty, patch configurations, or structural firewall strictness.

3. Custom System Graph Adjacency Mapping

Your specific network architecture maps security dependencies using these explicit connections:

- **Asset A (Attacker Entrance Node)** $\rightarrow$ B (Path Cost: 2), C (Path Cost: 4)
- **Asset B** $\rightarrow$ A (Path Cost: 2), D (Path Cost: 3), F (Path Cost: 5)
- **Asset C** $\rightarrow$ A (Path Cost: 4), D (Path Cost: 2), H (Path Cost: 6)
- **Asset D** $\rightarrow$ B (Path Cost: 3), C (Path Cost: 2), E (Path Cost: 2)
- **Asset E (Target Goal Node)** $\rightarrow$ D (Path Cost: 2), G (Path Cost: 3), H (Path Cost: 5)
- **Asset F** $\rightarrow$ B (Path Cost: 5), G (Path Cost: 4)
- **Asset G** $\rightarrow$ F (Path Cost: 4), E (Path Cost: 3)
- **Asset H** $\rightarrow$ C (Path Cost: 6), E (Path Cost: 5)

UNINFORMED SEARCH STRATEGIES

4. Uninformed Search Strategies

Uninformed (blind) search algorithms traverse the state space with no structural estimation of how close they are to the goal state.

A. Breadth-First Search (BFS)

- **Mechanism:** Explores nodes level-by-level using a First-In-First-Out (FIFO) queue data structure.
- **Behavior:** It guarantees finding the shallowest path containing the fewest total edge transitions. However, because it is blind to weight fields, it evaluates path length solely on step depth, making it suboptimal for weighted network architectures.
- **Simulation Result:** Discovers path A → B → D → E containing 3 transitions.

B. Depth-First Search (DFS)

- **Mechanism:** Penetrates deeply down a singular branch using a Last-In-First-Out (LIFO) stack layout before executing a backtrack step.
- **Behavior:** DFS exhibits low memory requirements but frequently wanders down inefficient paths, often settling on highly expensive routes.
- **Simulation Result:** Traverses the costlier alternative route A → C → H → E.

INFORMED SEARCH STRATEGIES

5. Informed Search Strategies

Informed search use problem-specific domain knowledge to guide path discovery efficiently.

A. Uniform Cost Search (UCS)

- **Mechanism:** Expands states based on lowest cumulative cost $g(n)$ from the start node using a Priority Queue.
- **Behavior:** UCS guarantees finding the mathematically cheapest cost vector across weighted topologies, independently of path layer counts.
- **Simulation Result:** Resolves path A → B → D → E with an optimal cost of 7.

B. A^* Search Algorithm

- **Mechanism:** Computes node value calculations via an evaluation function:

$$f(n) = g(n) + h(n)$$

where $g(n)$ represents the accumulated path cost from the origin, and $h(n)$ is a heuristic estimate forecasting the distance remaining to reach target E.

Heuristic (h) Estimation Table

The heuristic tracking vector uses these estimated lower-bound entries:

- **Node A:** 4 | **Node B:** 3 | **Node C:** 2 | **Node D:** 1
- **Node F:** 4 | **Node G:** 3 | **Node H:** 5 | **Node E (Goal):** 0

LOCAL & ADVERSARIAL SEARCH LAYERS

6. Simple Hill Climbing

- **Mechanism:** A local search method that moves to adjacent neighbors if they improve the current heuristic value.
- **The Local Minima Limitation:** Hill Climbing is vulnerable to structural traps where neighboring nodes return worse scores than the current node. If an architecture contains plateau configurations, the search stalls and marks the traversal as failed.

7. Adversarial Game Search Modeling

This section models defensive routing interactions as a competitive game.

- **Minimax Simulator:** Evaluates interactions assuming the Attacker acts to maximize compromise impacts, while an automated intrusion defense protocol serves as a Minimizing player trying to limit access risks.
- **Alpha-Beta Pruning Optimization:** Dynamically clips unpromising exploration paths when an alternative branch is already known to produce better outcomes. This minimizes node evaluations while matching standard Minimax decisions.

EMPIRICAL PERFORMANCE MATRIX

8. Simulation Run Results

This analytics data table summarizes the runtime performance characteristics computed from your custom network simulation targeting Goal Node E:

Algorithm Applied	Traversal Attack Path Calculated	Cumulative Step Cost	Total Graph Nodes Expanded
Breadth-First Search (BFS)	A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ E	7	4
Depth-First Search (DFS)	A $\rightarrow$ C $\rightarrow$ H $\rightarrow$ E	15	4
Uniform Cost Search (UCS)	A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ E	7	4
A* Search Algorithm	A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ E	7	4
Simple Hill Climbing	A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ E	7	4
Minimax Game Model	A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ E	7	6
Alpha-Beta Pruning	A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ E	7	4

DISCUSSION & CONCLUSION

9. Comparative Discussion & Analysis

- **Path Optimality:** UCS and A^* successfully found the most cost-effective path (A → B → D → E) with an optimal cost of 7. DFS performed poorly, highlighting a costly route of 15.
- **Resource Optimization:** While UCS and A^* found matching paths on this graph layout, A^* scales much better on larger production networks. Its heuristic function helps it bypass irrelevant sections of the search space.
- **Adversarial Evaluation:** Alpha-Beta Pruning found the exact same path as standard Minimax while reducing node expansions from 6 down to 4. This confirms its value for modeling defensive response engines.

10. Conclusion

This project successfully demonstrates graph-based threat path prediction using multiple AI search strategies. By analyzing the trade-offs between path cost, node expansion, and execution times, security teams can pinpoint vulnerable infrastructure combinations and patch systems before an incident occurs.